
PYTHON LOGIC BUILDING

Zero to Advanced

Learn How to Think Like a Programmer

Problem → Understanding → Algorithm → Pseudocode → Dry Run → Code

- Coding se pehle Thinking sikhayenge
- Har problem: Logic → Pseudocode → Dry Run → Code
 - Beginner se Advanced tak, ek connected roadmap
 - Practice Questions, Debugging aur Real Projects

Ek Hinglish, Beginner-Friendly Logic Building Guide

Table of Contents

1. Logic Kya Hai? (Foundation)

Programming logic, syntax se kaise alag hai, real-life examples

2. Problem Solving ka 14-Step Process

Har problem ko systematically solve karne ka fixed process

3. LEVEL 1 — Beginner Logic Building

Variables, Conditions, Comparison & Decision Making (10 problems)

4. LEVEL 2 — Loops & Pattern Logic

while/for, counters, accumulators, number problems, star patterns

5. LEVEL 3 — Intermediate Logic (Strings, Lists, Dicts)

Traversal, frequency, searching, sorting logic

6. LEVEL 4 — Functions & Modular Thinking

Bade problem ko chhote functions mein todna — Expense Tracker

7. LEVEL 5 — Real-World Project Problem Solving

ATM Simulation — pura project logic-first approach se

8. LEVEL 6 — Advanced Logic Building

Big O, Two Pointers, Sliding Window, Recursion, DP, Binary Search

9. LEVEL 7 — Data Structure Selection Thinking

List vs Set vs Dict vs Stack vs Queue — kab kya use karein

10. LEVEL 8 — Universal Problem-Solving Framework

12-step framework, kisi bhi problem par apply karo

11. Dry Run Training

Step-by-step table dry run practice

12. Logic Without Code

Sirf sochna — code likhne ki permission nahi

13. Debugging & Logic Errors

Syntax, Runtime, Logical errors — example bugs

14. Final Projects (Beginner → Advanced)

10+ project ideas with structured breakdown

15. 15-Day Logic Building Plan

Daily structured roadmap

16. Final Problem-Solving Checklist

Print karke apne desk par lagao

FOUNDATION

Logic Kya Hai?

Sabse pehle samjhein — code se pehle sochna

Programming Logic matlab kisi problem ko solve karne ka **step-by-step tareeka (sequence of steps)** sochna — bina kisi specific programming language ke. Jab aap decide karte ho ki 'pehle ye karna hai, phir ye, agar ye condition true hai to ye karna hai' — wahi logic hai.

Logic vs Syntax — Fark Kya Hai?

Aspect	Logic	Syntax
Matlab	Problem solve karne ka soch (WHAT to do)	Language ke rules (HOW to write)
Depend karta hai	Aapki thinking par	Python ke grammar rules par
Language-independent?	Haan — same logic C, Java, Python sabme chalta hai	Nahi — har language ka syntax alag hai
Seekhne mein mushkil	Zyada — practice se aata hai	Kam — rules yaad ho jaate hain
Example	"Agar number 2 se divide ho jaaye to even hai"	if num % 2 == 0:

Bahut se beginners **syntax yaad karte hain lekin logic nahi samajhte** — isliye jab thoda naya problem aata hai, wo stuck ho jaate hain. Is guide mein hum har jagah **pehle logic**, phir syntax sikhayenge.

Real-Life Example se Samjho

Socho aap **chai bana rahe ho**. Aap directly chai nahi bana dete — pehle soch ke steps banate ho:

1. Pani lo (Input)
2. Pani ko garam karo (Process)
3. Agar pani ubal raha hai to chai patti daalo (Condition)
4. Doodh aur cheeni daalo (Process)
5. 2 minute ubalne do (Loop/Wait)
6. Cup mein nikalo (Output)

Yehi programming logic hai — **Input lo** → **Conditions check karo** → **Steps follow karo** → **Output do**. Python code sirf inhi steps ko computer ki language mein likhna hai.

Har Problem Ko Is Format Mein Sochna Seekho

Step	Kya Karna Hai
1. Problem	Kya solve karna hai, clearly likho
2. Understanding	Apni language mein problem explain karo
3. Input	Konsi values di gayi hain?
4. Output	Konsi value chahiye result mein?

Step	Kya Karna Hai
5. Conditions	Kaunse rules/checks lagenge?
6. Steps	Kaunse operations, kis order mein?
7. Algorithm	Steps ko ek clear sequence mein likho
8. Pseudocode	Algorithm ko code-jaisi English mein likho
9. Dry Run	Manually values daal ke check karo ki logic sahi hai
10. Code	Ab Python mein likho

PROCESS

Problem Solving ka 14-Step Process

Har programming problem ke liye fixed process

1. **Problem ko carefully read karna** — Jaldi mat karo. Poora problem statement 2 baar padho.
2. **Apni language mein samajhna** — “Iska matlab hai ki mujhe...” bol ke khud ko explain karo.
3. **Input identify karna** — Kya values di gayi hain? Kitni? Konsa data type?
4. **Expected output identify karna** — Result kaisa dikhna chahiye? Number? String? List?
5. **Conditions identify karna** — Kaunse special cases/rules hain?
6. **Problem ko chhote parts mein todna** — Bada problem → 2-3 chhote sub-problems
7. **Required variables identify karna** — Kaunsi values store karni padengi?
8. **Algorithm banana** — Steps ka logical sequence
9. **Pseudocode likhna** — English/Hinglish mein code-jaisa structure
10. **Dry run karna** — Table bana ke manually values check karo
11. **Python code likhna** — Ab syntax use karo
12. **Code ko test karna** — Normal inputs se test karo
13. **Edge cases check karna** — Empty input, negative number, zero, bahut bada number
14. **Code ko optimize karna** — Kya isse fast/short/cleaner likha ja sakta hai?

Example: Isi Process Ko Ek Chhote Problem Par Apply Karte Hain

Problem: Do numbers ka sum print karo.

Step	Answer
Understanding	Mujhe 2 numbers add karke result dikhana hai
Input	num1, num2 (dono integers)
Output	num1 + num2 ka result
Conditions	Koi special condition nahi
Variables	num1, num2, total
Algorithm	1) num1 lo 2) num2 lo 3) total = num1+num2 4) total print karo

```
# Pseudocode:
START
  READ num1
  READ num2
  total = num1 + num2
  PRINT total
END

# Python Code:
num1 = 5
num2 = 3
total = num1 + num2
print(total)  # Output: 8
```


LEVEL 1

Beginner Logic Building

Variables, Conditions, Comparison & Decision Making

A. Variables & Basic Thinking

Variable ek **naam wala box** hai jisme aap koi value store karte ho. Jab bhi problem solve karo, sabse pehle socho: “**Kaunsi values mujhe yaad rakhni padengi?**” — wahi tumhare variables hain.

- **Variable identify karne ka tarika:** Problem mein jo bhi ‘changing ya storable’ cheez dikhe (naam, age, total, count) — wo variable hai
- **Kaunsi value store karni hai:** Input se aayi value, ya calculation ka result
- **Input/Output thinking:** Hamesha socho — “User kya dega (input)?” aur “Mujhe kya dikhana hai (output)?”
- **Simple calculations:** +, -, *, /, // (integer division), % (remainder), ** (power)

B. Conditions — Decision Making

Conditions se hum computer ko decide karna sikhate hain: “**Agar ye sach hai to ye karo, warna wo karo.**”

```
if condition:
    # jab condition True ho
elif another_condition:
    # jab pehli False, ye True ho
else:
    # jab sab False ho

# Nested if -> if ke andar if
# AND -> dono condition true honi chahiye
# OR -> koi ek bhi true ho to chalega
# NOT -> condition ko ulta kar deta hai
```

C. Comparison & Decision Making — Worked Problems

Problem 1: Number Positive, Negative ya Zero?

PROBLEM

Ek number diya gaya hai. Check karo ki wo positive hai, negative hai, ya zero hai.

THINKING

- Number decimal ho sakta hai kya?
- Zero ko positive maanein ya alag case?

LOGIC

- 3 possibilities hain: >0, <0, ==0
- Ek if-elif-else structure kaafi hai

PSEUDOCODE

```

START
  READ num
  IF num > 0:
    PRINT "Positive"
  ELIF num < 0:
    PRINT "Negative"
  ELSE:
    PRINT "Zero"
END

```

DRY RUN

num	Check	Result
-5	num > 0? No, num < 0? Yes	Negative
0	num>0? No, num<0? No	Zero
7	num>0? Yes	Positive

PYTHON CODE

```

num = -5
if num > 0:
    print("Positive")
elif num < 0:
    print("Negative")
else:
    print("Zero")

```

EXPLANATION

Yahan order important hai — pehle >0 check kiya, phir <0, jo bache wo automatically 0 hai.

Common Mistakes:

- Sirf if-if-if likhna (elif nahi) — isse multiple prints ho sakte hain agar galti se overlap ho

Problem 2: Even ya Odd Number?

PROBLEM

Ek number diya gaya hai, check karo even hai ya odd.

THINKING

- Negative number ke liye bhi even/odd same rule chalega kya?
- 0 even hota hai ya odd?

LOGIC

- Number ko 2 se divide karke remainder (%) dekho
- Agar remainder 0 hai → Even, warna Odd

PSEUDOCODE

```

START
  READ num
  IF num % 2 == 0:
    PRINT "Even"
  ELSE:
    PRINT "Odd"
END

```

DRY RUN

num	num % 2	Result
10	0	Even
7	1	Odd
0	0	Even

PYTHON CODE

```

num = 7
if num % 2 == 0:
    print("Even")
else:
    print("Odd")

```

EXPLANATION

% operator hume division ka **remainder** deta hai. Har even number 2 se pura divide hota hai, isliye remainder 0 hota hai.

Common Mistakes:

- num % 2 == 0 ki jagah num / 2 == 0 likh dena — galat, kyunki / division deta hai remainder nahi

Problem 3: Largest of Three Numbers

PROBLEM

Teen numbers diye gaye hain, sabse bada number find karo.

THINKING

- Kya do numbers equal ho sakte hain?
- Kya sirf comparison se kaam ho jayega, ya sort karna padega?

LOGIC

- Pehle a aur b compare karo
- Fir jo bada hai use c ke saath compare karo
- Nested if ya multiple conditions dono chal sakte hain

PSEUDOCODE

```

START
  READ a, b, c
  IF a >= b AND a >= c:
    largest = a
  ELIF b >= a AND b >= c:
    largest = b
  ELSE:
    largest = c
  PRINT largest
END

```

DRY RUN

a	b	c	Check	largest
10	25	15	b>=a and b>=c? Yes	25
8	8	3	a>=b and a>=c? Yes	8

PYTHON CODE

```

a, b, c = 10, 25, 15
if a >= b and a >= c:
    largest = a
elif b >= a and b >= c:
    largest = b
else:
    largest = c
print(largest)

```

EXPLANATION

Har condition mein hum ek number ko dono doosre numbers se compare karte hain — tabhi confirm hota hai wo sabse bada hai.

Common Mistakes:

- Sirf a>b and a>c check karna aur >= na lagana — equal values wale case mein galat result dega

Problem 4: Leap Year Check

PROBLEM

Ek year diya gaya hai, check karo wo leap year hai ya nahi.

THINKING

- Leap year ka exact rule kya hai?
- Kya har 4-divisible year leap hota hai?

LOGIC

- Rule: Year 4 se divisible ho **AND** (100 se divisible NA ho **OR** 400 se divisible ho)
- Isliye 2000 leap hai lekin 1900 leap nahi hai

PSEUDOCODE

```

START
  READ year
  IF year % 4 == 0 AND (year % 100 != 0 OR year % 400 == 0):
    PRINT "Leap Year"
  ELSE:
    PRINT "Not a Leap Year"
END

```

DRY RUN

year	%4==0	%100!=0	%400==0	Result
2000	True	False	True	Leap Year
1900	True	False	False	Not Leap
2024	True	True	-	Leap Year

PYTHON CODE

```

year = 2000
if year % 4 == 0 and (year % 100 != 0 or year % 400 == 0):
    print("Leap Year")
else:
    print("Not a Leap Year")

```

EXPLANATION

Ye classic example hai jahan sirf ek condition (%4==0) kaafi nahi hai — century years ke liye extra rule chahiye.

Common Mistakes:

- Sirf `year % 4 == 0` check karke reh jaana — 1900 jaisa case galat handle hoga

Practice Questions — Level 1 (Conditions)

EASY

1. Largest of two numbers
2. Voting eligibility (age ≥ 18 ?)
3. Simple calculator (+, -, *, /)

MEDIUM

1. Pass/Fail based on marks
2. Grade calculator (A/B/C/D/F using elif ladder)
3. Login validation (username & password match)

HARD

1. Grade calculator jisme attendance bhi factor ho
2. Number ko 'Positive Even', 'Negative Odd' jaise combined labels do

Yaad rakhna: pehle khud solve karne ki koshish karo, seedha solution mat dekho!

LEVEL 2

Loops & Pattern Logic

Repetition ko thinking se samjho, sirf syntax se nahi

Loop Kab Aur Kaise Use Karna Hai?

- **Kab loop use karna hai?** Jab bhi koi kaam *baar-baar* repeat karna ho (print karna, add karna, check karna)
- **Kitni baar chalega?** Agar exact count pata hai → **for** loop. Agar condition based hai (pata nahi kitni baar) → **while** loop
- **Starting value kaise decide karein?** Problem mein diya starting point (jaise 1 se N tak → start=1)
- **Ending condition kaise decide karein?** Jahan tak loop chalna chahiye, waha loop rukna chahiye (jaise $i \leq N$)
- **Counter** — ek variable jo loop ke har round mein badhta/ghatta hai ($i, i+1, i+2...$)
- **Accumulator** — ek variable jo values ko collect/add karta jaata hai ($total = total + i$)

```
# for loop -> jab count pata ho
for i in range(1, 6):      # 1,2,3,4,5
    print(i)

# while loop -> jab condition based ho
i = 1
while i <= 5:
    print(i)
    i += 1                # counter update -- yaad rakhna warna infinite loop!

# break -> loop turant rok do
# continue -> is iteration ko skip karke agle round pe jao
```

Number Logic — Worked Problems

Problem 5: Sum of First N Numbers

PROBLEM

1 se N tak sabhi numbers ka sum nikalo.

THINKING

- N kya 0 ya negative ho sakta hai?
- Loop kitni baar chalega?

LOGIC

- Ek accumulator variable $total=0$ banao
- 1 se N tak loop chalao, har number ko total mein add karo

PSEUDOCODE

```

START
  READ N
  total = 0
  FOR i FROM 1 TO N:
    total = total + i
  PRINT total
END

```

DRY RUN

i	total (before)	calculation	total (after)
1	0	0+1	1
2	1	1+2	3
3	3	3+3	6
4	6	6+4	10
5	10	10+5	15

PYTHON CODE

```

N = 5
total = 0
for i in range(1, N+1):
    total += i
print(total) # Output: 15

```

EXPLANATION

`range(1, N+1)` isliye kyunki range ka last number exclude hota hai — N ko include karne ke liye N+1 likhna padta hai.

Common Mistakes:

- `range(1, N)` likh dena — last number N miss ho jayega

Problem 6: Factorial of a Number

PROBLEM

N! (factorial) nikalo, yaani $N * (N-1) * (N-2) * \dots * 1$

THINKING

- 0! ka value kya hota hai? (Special case: $0! = 1$)
- Negative number ke liye factorial exist karta hai?

LOGIC

- Accumulator ki starting value 1 honi chahiye (0 nahi, kyunki multiplication hai)
- 1 se N tak har number ko multiply karo

PSEUDOCODE

```

START
  READ N
  fact = 1
  FOR i FROM 1 TO N:
    fact = fact * i
  PRINT fact
END

```

DRY RUN

i	fact (before)	calc	fact (after)
1	1	1*1	1
2	1	1*2	2
3	2	2*3	6
4	6	6*4	24

PYTHON CODE

```

N = 4
fact = 1
for i in range(1, N+1):
    fact *= i
print(fact) # Output: 24

```

EXPLANATION

Multiplication mein accumulator hamesha 1 se start hota hai (agar 0 se start karoge to sab kuch 0 ban jayega).

Common Mistakes:

- fact = 0 se start karna — sabse common mistake, poora result 0 aa jaata hai

Problem 7: Reverse a Number

PROBLEM

Ek number diya hai (jaise 1234), use reverse karo (4321).

THINKING

- Number negative ho to kya karein?
- Trailing zero wale number (jaise 120) reverse hone par kya hoga?

LOGIC

- Last digit nikalne ke liye % 10 use karo
- Us digit ko reversed number mein add karo (reversed = reversed*10 + digit)
- Original number ko //10 se chhota karte jao

PSEUDOCODE

```

START
READ num
reversed = 0
WHILE num > 0:
    digit = num % 10
    reversed = reversed * 10 + digit
    num = num // 10
PRINT reversed
END

```

DRY RUN

num	digit	reversed (calc)	num after //10
1234	4	$0 \times 10 + 4 = 4$	123
123	3	$4 \times 10 + 3 = 43$	12
12	2	$43 \times 10 + 2 = 432$	1
1	1	$432 \times 10 + 1 = 4321$	0

PYTHON CODE

```

num = 1234
reversed_num = 0
while num > 0:
    digit = num % 10
    reversed_num = reversed_num * 10 + digit
    num = num // 10
print(reversed_num)  # Output: 4321

```

EXPLANATION

Har step mein last digit nikaal ke reversed number ke end mein jodte hain — yehi trick digit reversal ka core hai.

Common Mistakes:

- Variable ka naam 'reversed' rakhna (Python ka built-in reversed() function hai) — isliye reversed_num better hai

Problem 8: Palindrome Number Check

PROBLEM

Check karo number palindrome hai ya nahi (aage se padho ya peeche se, same lage — jaise 121, 1331).

THINKING

- Palindrome check karne ke liye kya poora reverse karna zaroori hai?

LOGIC

- Number ka reverse nikalo (jaisa Problem 7 mein kiya)
- Agar reverse == original number, to palindrome hai

PSEUDOCODE

```

START
  READ num
  original = num
  reversed = 0
  WHILE num > 0:
    digit = num % 10
    reversed = reversed*10 + digit
    num = num // 10
  IF reversed == original:
    PRINT "Palindrome"
  ELSE:
    PRINT "Not Palindrome"
END

```

DRY RUN

num	digit	reversed
121	1	1
12	2	12
1	1	121

PYTHON CODE

```

num = 121
original = num
reversed_num = 0
while num > 0:
    digit = num % 10
    reversed_num = reversed_num * 10 + digit
    num = num // 10

if reversed_num == original:
    print("Palindrome")
else:
    print("Not Palindrome")

```

EXPLANATION

Yahan hume original value **save karke rakhni** padi kyunki num ki value loop mein change ho jaati hai (0 tak).

Common Mistakes:

- original = num save na karna — phir compare karne ke liye kuch bachega hi nahi

Problem 9: Prime Number Check

PROBLEM

Check karo number prime hai ya nahi (sirf 1 aur khud se divisible ho).

THINKING

- 1 prime hota hai kya? (Nahi — special case)
- Kya sabhi numbers 2 se num-1 tak check karne zaroori hain?

LOGIC

- 1 aur usse chhote numbers ko directly 'Not Prime' maano
- 2 se num-1 tak loop chalao, agar koi bhi number num ko fully divide kare → Not Prime
- (Optimization: sirf sqrt(num) tak check karna kaafi hai — Level 6 mein dekhenge)

PSEUDOCODE

```
START
  READ num
  IF num <= 1:
    PRINT "Not Prime"
  ELSE:
    is_prime = True
    FOR i FROM 2 TO num-1:
      IF num % i == 0:
        is_prime = False
        BREAK
    PRINT "Prime" if is_prime else "Not Prime"
END
```

DRY RUN

i	num % i	is_prime
2	7%2=1	True (continue)
3	7%3=1	True (continue)
4	7%4=3	True (continue)
5,6	not 0	True → Prime

PYTHON CODE

```
num = 7
is_prime = True
if num <= 1:
    is_prime = False
else:
    for i in range(2, num):
        if num % i == 0:
            is_prime = False
            break

print("Prime" if is_prime else "Not Prime")
```

EXPLANATION

Break isliye use kiya kyunki ek bhi divisor mil jaaye to aage check karne ki zaroorat nahi — time bachta hai.

Common Mistakes:

- Loop ko range(2, num+1) tak le jaana — tab num khud se divisible hoga aur galat 'Not Prime' aa sakta hai

Practice Questions — Level 2 (Numbers)

EASY

1. 1 to N print karo
2. N to 1 print karo (reverse loop)
3. Even numbers 1 to N print karo

MEDIUM

1. Count digits of a number
2. Sum of digits of a number
3. Fibonacci series first N terms

HARD

1. Armstrong number check (jaise $153 = 1^3 + 5^3 + 3^3$)
2. Perfect number check (jaise $6 = 1 + 2 + 3$)

Yaad rakhna: pehle khud solve karne ki koshish karo, seedha solution mat dekho!

Pattern Problems — Rows aur Columns Ki Thinking

Pattern problems mein **outer loop** = rows control karta hai, aur **inner loop** = us row mein columns/stars print karta hai. Pattern dekh kar manually observe karo: "Row number 3 mein kitne stars hain? Kya wo row number ke barabar hai?" — wahi tumhara inner loop ka range banega.

Problem 10: Right Triangle Star Pattern

PROBLEM

N=4 ke liye ye pattern print karo: * ** *** ****

LOGIC

- Outer loop rows ke liye: $i = 1$ to N
- Row i mein exactly i stars hain — to inner loop: $j = 1$ to i
- Har inner loop ke baad newline print karo

PSEUDOCODE

```
START
  FOR i FROM 1 TO N:
    FOR j FROM 1 TO i:
      PRINT "*" (no newline)
    PRINT newline
END
```

DRY RUN

i (row)	Inner loop j	Output row
1	j=1	*
2	j=1,2	**
3	j=1,2,3	***
4	j=1..4	****

PYTHON CODE

```
N = 4
for i in range(1, N+1):
    for j in range(1, i+1):
        print("*", end="")
    print()
```

EXPLANATION

end="" isliye use kiya taaki har star same line mein print ho, print() khali call se sirf newline aata hai row khatam hone par.

Common Mistakes:

- Inner loop ko range(1, N+1) rakh dena (i ki jagah) — tab har row mein N stars aa jayenge, triangle nahi banega

Problem 11: Number Pyramid Pattern

PROBLEM

N=4 ke liye ye pattern print karo: 1 12 123 1234

LOGIC

- Ye Right Triangle jaisa hi hai, bas '*' ki jagah 'j' (number) print karna hai

PSEUDOCODE

```
START
FOR i FROM 1 TO N:
    FOR j FROM 1 TO i:
        PRINT j (no newline)
    PRINT newline
END
```

PYTHON CODE

```
N = 4
for i in range(1, N+1):
    for j in range(1, i+1):
        print(j, end="")
    print()
```

EXPLANATION

Pattern problems mein sirf ye badalta hai ki inner loop **kya** print kar raha hai (star, number, ya letter) — loop ka structure same rehta hai.

Practice Questions — Level 2 (Patterns)

EASY

1. Inverted right triangle (stars kam hote jaayein)
2. Square pattern of stars

MEDIUM

1. Pyramid pattern (spaces + stars, centered)
2. Diamond pattern

HARD

1. Number triangle jisme har row mein numbers repeat ho (1, 22, 333)
2. Floyd's Triangle

Yaad rakhna: pehle khud solve karne ki koshish karo, seedha solution mat dekho!

LEVEL 3

Intermediate Logic

Strings, Lists, Tuples, Sets, Dictionaries

Ab hum real problem-solving ki taraf badhte hain. Yahan se pehle main tumse **questions poochunga**, taaki tum khud sochne ka chance lo, uske baad hi logic develop karenge.

Example: Second Largest Number — Sochne Ka Tarika

Problem: Ek list mein se second largest number find karo.

Code likhne se pehle khud se ye sawaal poocho:

- Kya list empty ho sakti hai? (Agar haan — handle karna padega)
- Duplicate values allowed hain kya? [5,5,5] mein second largest kya hoga?
- Kya ek hi largest value ho sakti hai? Agar sab numbers same hon to?
- 'Second largest' ka matlab hai — largest ko chhodkar, jo agla sabse bada hai (duplicate largest ignore karke)

Ab in sawaalon ke jawab se hum logic develop karte hain — do variables rakhenge: **largest** aur **second_largest**.

Problem 12: Second Largest Number in a List

PROBLEM

List [10, 25, 8, 25, 19] mein se second largest number find karo.

THINKING

- Duplicates ko kaise handle karein (25, 25 hai list mein)?

LOGIC

- largest aur second_largest ko bahut chhoti value se initialize karo
- List traverse karo: agar number > largest → second_largest = largest, largest = number
- Warna agar number > second_largest AND number != largest → second_largest = number

PSEUDOCODE

```
START
largest = -infinity
second_largest = -infinity
FOR num IN list:
    IF num > largest:
        second_largest = largest
        largest = num
    ELIF num > second_largest AND num != largest:
        second_largest = num
PRINT second_largest
END
```

DRY RUN

num	largest (before)	second (before)	largest (after)	second (after)
10	-inf	-inf	10	-inf
25	10	-inf	25	10

num	largest (before)	second (before)	largest (after)	second (after)
8	25	10	25	10
25	25	10	25	10 (skip, duplicate)
19	25	10	25	19

PYTHON CODE

```
nums = [10, 25, 8, 25, 19]
largest = float('-inf')
second_largest = float('-inf')

for num in nums:
    if num > largest:
        second_largest = largest
        largest = num
    elif num > second_largest and num != largest:
        second_largest = num

print(second_largest) # Output: 19
```

EXPLANATION

num != largest check zaroori hai warna duplicate largest (25) galti se second_largest ban jaata.

Common Mistakes:

- Sirf list ko sort karke -2 index le lena — duplicates ke case mein galat answer dega (jab tak set() na use karo)

Problem 13: Reverse a String

PROBLEM

Ek string ko reverse karo ('hello' → 'olleh').

THINKING

- Kya Python ka slicing use kar sakte hain, ya khud loop se banana hai?

LOGIC

- String ke characters ko peeche se aage collect karo, ya slicing[::-1] use karo

PSEUDOCODE

```
START
READ text
reversed_text = ""
FOR i FROM len(text)-1 DOWNT0 0:
    reversed_text = reversed_text + text[i]
PRINT reversed_text
END
```

DRY RUN

i	text[i]	reversed_text
4	'o'	'o'

i	text[i]	reversed_text
3	'l'	'ol'
2	'l'	'oll'
1	'e'	'olle'
0	'h'	'olleh'

PYTHON CODE

```
text = "hello"
reversed_text = ""
for i in range(len(text)-1, -1, -1):
    reversed_text += text[i]
print(reversed_text)

# Shortcut using slicing:
print(text[::-1]) # Same output: olleh
```

EXPLANATION

Loop-wala tareeka logic samajhne ke liye best hai, lekin real code mein text[::-1] slicing zyada common hai.

Problem 14: Character Frequency Counter

PROBLEM

String mein har character kitni baar aaya hai, count karo.

THINKING

- Space aur uppercase/lowercase ko alag character maanein ya same?

LOGIC

- Ek empty dictionary banao
- Har character ke liye: agar dictionary mein hai to count+1, warna naya entry (count=1) banao

PSEUDOCODE

```
START
freq = {}
FOR ch IN text:
    IF ch IN freq:
        freq[ch] = freq[ch] + 1
    ELSE:
        freq[ch] = 1
PRINT freq
END
```

DRY RUN

ch	freq (before)	freq (after)
'a'	{}	{'a':1}
'p'	{'a':1}	{'a':1,'p':1}

ch	freq (before)	freq (after)
'p'	{'a':1,'p':1}	{'a':1,'p':2}

PYTHON CODE

```
text = "apple"
freq = {}
for ch in text:
    if ch in freq:
        freq[ch] += 1
    else:
        freq[ch] = 1
print(freq)  # {'a':1, 'p':2, 'l':1, 'e':1}

# Shortcut using Counter:
from collections import Counter
print(Counter(text))
```

EXPLANATION

Dictionary is tarah ke 'counting' problems ke liye perfect data structure hai kyunki key-value pairs mein directly count store ho jaata hai.

Problem 15: Remove Duplicates from a List

PROBLEM

List [1,2,2,3,4,4,5] se duplicates hata kar unique list banao, order maintain karte hue.

THINKING

- Kya order maintain karna zaroori hai? (Set use karne se order kabhi kabhi badal jaata hai)

LOGIC

- Ek empty result list aur ek 'seen' set banao
- Har element ke liye: agar seen mein nahi hai to result mein add karo aur seen mein daalo

PSEUDOCODE

```
START
result = []
seen = {} (empty set)
FOR num IN list:
    IF num NOT IN seen:
        result.append(num)
        seen.add(num)
PRINT result
END
```

DRY RUN

num	seen (before)	action
1	{}	add → result=[1]
2	{1}	add → result=[1,2]

num	seen (before)	action
2	{1,2}	skip (already seen)
3	{1,2}	add → result=[1,2,3]

PYTHON CODE

```
nums = [1, 2, 2, 3, 4, 4, 5]
result = []
seen = set()
for num in nums:
    if num not in seen:
        result.append(num)
        seen.add(num)
print(result)  # [1, 2, 3, 4, 5]
```

EXPLANATION

Set use karne se 'already seen hai ya nahi' check $O(1)$ time mein hota hai — list mein 'in' check karna slow hota hai bade data ke liye.

Practice Questions — Level 3

EASY

1. Word counting in a sentence
2. Find maximum/minimum in a list
3. Palindrome string check

MEDIUM

1. Anagram check (do words same letters se bane hain kya)
2. Find common elements in two lists
3. Find missing number in a sequence 1 to N

HARD

1. Find all duplicate elements in a list
2. Sorting logic — bubble sort khud implement karo
3. Linear search vs Binary search khud likho

Yaad rakhna: pehle khud solve karne ki koshish karo, seedha solution mat dekho!

LEVEL 4

Functions & Modular Thinking

Bade problem ko chhote, reusable pieces mein todna

Function Kya Hai Aur Kab Banana Chahiye?

- **Function** = code ka ek reusable block jo ek specific kaam karta hai
- **Kab banana chahiye?** Jab koi kaam *baar-baar* repeat ho raha ho, ya jab ek bada problem ke *alag-alag* hisse hon
- **Parameters** = function ko diya gaya input
- **Arguments** = function call karte waqt di gayi actual values
- **Return** = function se wapas mila result
- **Function composition** = ek function ke andar doosra function call karna

```
def add(a, b):          # a, b -> parameters
    return a + b

result = add(5, 3)      # 5, 3 -> arguments, result = 8
```

Example Project: Expense Tracker — Modular Thinking

Problem: User apne daily expenses add kare, unhe dekh sake, total nikal sake, category-wise summary dekh sake, aur expense delete bhi kar sake.

Pehle poore problem ko chhote functions mein divide karte hain (architecture pehle, code baad mein):

```
expenses = [] # sabhi expenses is list mein store honge (har expense ek dictionary)

add_expense()      # naya expense add karna
view_expenses()    # sabhi expenses dikhana
calculate_total()   # total expense nikalna
category_summary()  # category-wise total dikhana
delete_expense()    # ek expense delete karna
```

Har function ka kaam:

Function	Input	Kaam	Output
add_expense()	amount, category, note	Dictionary bana ke list mein append	Updated list
view_expenses()	koi nahi	List ko loop karke print	Screen par saari entries
calculate_total()	koi nahi	Sab amounts ko sum karna	Total (number)
category_summary()	koi nahi	Category ke hisaab se group karke sum	Dictionary {category: total}
delete_expense()	index/id	List se ek entry remove karna	Updated list

```

expenses = []

def add_expense(amount, category, note=""):
    expenses.append({"amount": amount, "category": category, "note": note})

def view_expenses():
    for e in expenses:
        print(e["category"], "-", e["amount"], "-", e["note"])

def calculate_total():
    total = 0
    for e in expenses:
        total += e["amount"]
    return total

def category_summary():
    summary = {}
    for e in expenses:
        cat = e["category"]
        summary[cat] = summary.get(cat, 0) + e["amount"]
    return summary

def delete_expense(index):
    if 0 <= index < len(expenses):
        expenses.pop(index)

# Example usage:
add_expense(200, "Food", "Lunch")
add_expense(500, "Travel", "Bus pass")
view_expenses()
print("Total:", calculate_total())
print("Summary:", category_summary())

```

Notice karo — har function **ek hi kaam** karta hai (Single Responsibility). Isse code padhna, test karna aur bug dhundna bahut aasan ho jaata hai.

Practice Questions — Level 4

EASY

1. Function banao jo do numbers ka average nikale
2. Function banao jo string ko uppercase kare

MEDIUM

1. Contact Book ke liye functions design karo (add_contact, search_contact, delete_contact)
2. Number ki list ko function ke through sort karo (khud ka sorting logic)

HARD

1. Poora To-Do List app functions mein todo (add_task, complete_task, view_tasks, delete_task)

Yaad rakhna: pehle khud solve karne ki koshish karo, seedha solution mat dekho!

LEVEL 5

Real-World Project: ATM Simulation

Logic-first approach se poora project banate hain

Is level mein hum ek real project **step-by-step** banayenge — Requirements se leke Code tak, poora process follow karte hue.

- 1. Requirements:** User PIN check kar sake, balance dekh sake, paisa deposit/withdraw kar sake, transaction history dekh sake.
- 2. Problem Breakdown:** 4 chhote features: (a) PIN Authentication (b) Check Balance (c) Deposit (d) Withdraw
- 3. Data Required:** balance (number), correct_pin (number), transaction_history (list)
- 4. Functions Required:** check_pin(), show_balance(), deposit(amount), withdraw(amount), show_history()
- 5. Algorithm:** PIN maango → sahi hai to menu dikhao → user ka choice lo → respective function call karo → loop chalate raho jab tak Exit na chune

Pseudocode

```
START
    balance = 1000
    correct_pin = 1234
    history = []

    READ entered_pin
    IF entered_pin != correct_pin:
        PRINT "Wrong PIN" and STOP

    LOOP forever:
        SHOW menu (1-Balance, 2-Deposit, 3-Withdraw, 4-History, 5-Exit)
        READ choice
        IF choice == 1: PRINT balance
        IF choice == 2: READ amount; balance += amount; history.append("Deposit: "+amount)
        IF choice == 3:
            IF amount <= balance: balance -= amount; history.append("Withdraw: "+amount)
            ELSE: PRINT "Insufficient balance"
        IF choice == 4: PRINT history
        IF choice == 5: BREAK

    END
```

Dry Run (Sample Flow)

Action	balance (before)	Calculation	balance (after)
Start	1000	-	1000
Deposit 500	1000	1000+500	1500
Withdraw 700	1500	1500-700	800
Withdraw 2000	800	2000 > 800 → rejected	800 (no change)

Python Code

```

balance = 1000
correct_pin = 1234
history = []

entered_pin = int(input("Enter PIN: "))

if entered_pin != correct_pin:
    print("Wrong PIN")
else:
    while True:
        print("\n1. Check Balance  2. Deposit  3. Withdraw  4. History  5. Exit")
        choice = int(input("Choose an option: "))

        if choice == 1:
            print("Balance:", balance)

        elif choice == 2:
            amount = float(input("Enter deposit amount: "))
            balance += amount
            history.append(f"Deposited {amount}")

        elif choice == 3:
            amount = float(input("Enter withdraw amount: "))
            if amount <= balance:
                balance -= amount
                history.append(f"Withdrew {amount}")
            else:
                print("Insufficient balance")

        elif choice == 4:
            for h in history:
                print(h)

        elif choice == 5:
            print("Thank you!")
            break

        else:
            print("Invalid choice")

```

Testing, Edge Cases & Improvements

- **Testing:** Normal deposit/withdraw try karo, sahi/galat PIN dono try karo
- **Edge cases:** Negative amount deposit karna, withdraw amount = 0, balance exactly equal to withdraw amount
- **Improvements:** 3 galat PIN attempts ke baad account lock karna, amount ko negative allow na karna, file mein data save karna (persistence)

Doosre Real-World Projects (Brief Structure)

Inhe bhi isi 10-step process se khud banane ki koshish karo:

Project	Core Data	Key Functions
Number Guessing Game	secret_number, attempts	generate_number(), check_guess(), give_hint()
Quiz Application	questions list, score	ask_question(), check_answer(), show_score()
Contact Book	contacts list/dict	add_contact(), search_contact(), delete_contact()
Student Management	students dict	add_student(), update_marks(), calculate_average()
Inventory Management	items dict (name:qty,price)	add_item(), update_stock(), low_stock_alert()
To-Do List	tasks list	add_task(), complete_task(), view_pending()
Password Validation	password string	check_length(), check_special_char(), check_strength()

LEVEL 6

Advanced Logic Building

Complexity, Optimization aur Powerful Techniques

Time & Space Complexity (Big O)

Big O batata hai ki input badhne par tumhara code **kitna slow** hota hai. Ye **exact time nahi**, balki growth rate batata hai.

Notation	Naam	Example
$O(1)$	Constant	list[0] access karna
$O(\log n)$	Logarithmic	Binary Search
$O(n)$	Linear	Ek loop se list traverse karna
$O(n \log n)$	Linearithmic	Merge Sort, Quick Sort (average)
$O(n^2)$	Quadratic	Do nested loops (jaise Bubble Sort)

Brute force thinking = sabse simple/direct solution jo kaam kare (chahe slow ho). Pehle brute force socho, phir **optimize** karna seekho.

Two Pointers

Jab sorted array/list mein **do positions se ek saath** kaam karna ho (ek shuru se, ek end se), tab Two Pointers use hota hai. Isse nested loop ka kaam single pass mein ho jaata hai.

```
# Problem: Sorted list mein pair find karo jiska sum = target
nums = [2, 4, 6, 8, 11]
target = 14
left, right = 0, len(nums)-1

while left < right:
    s = nums[left] + nums[right]
    if s == target:
        print(nums[left], nums[right]); break
    elif s < target:
        left += 1      # sum kam hai -> chhota pointer aage badhao
    else:
        right -= 1     # sum zyada hai -> bada pointer peeche lao
```

Sliding Window

Jab kisi **continuous sub-part** (subarray/substring) par kaam karna ho — jaise 'size K ka maximum sum subarray' — tab har baar poora naya loop chalane ke bajaye, ek 'window' slide karte hain.

```
# Problem: size-k window ka maximum sum nikalo
nums = [2, 1, 5, 1, 3, 2]
k = 3
window_sum = sum(nums[:k])
max_sum = window_sum

for i in range(k, len(nums)):
    window_sum += nums[i] - nums[i-k] # naya element add, purana hatao
    max_sum = max(max_sum, window_sum)

print(max_sum) # Output: 9
```

Hashing

Dictionary/Set use karke hum **O(1)** mein **lookup** kar sakte hain — “ye value pehle dekhi hai kya?” jaise sawaal turant answer ho jaate hain (jaisa Problem 12, 14, 15 mein dekha).

Recursion

Recursion matlab **function khud ko call kare**, chhote version of the same problem ke saath, jab tak ek ‘base case’ na aa jaaye (jahan recursion rukta hai).

```
def factorial(n):
    if n == 0: # base case -- yaha rukna hai
        return 1
    return n * factorial(n-1) # chhota problem: (n-1)!

print(factorial(5)) # 5*4*3*2*1 = 120

# Call stack:
# factorial(5) -> 5 * factorial(4)
#               -> 5 * (4 * factorial(3))
#               -> 5 * (4 * (3 * factorial(2)))
#               -> ... -> 5*4*3*2*1
```

Golden Rule: Har recursive function mein ek base case zaroor hona chahiye, warna infinite recursion → crash (RecursionError).

Backtracking

Backtracking matlab — ek path try karo, agar wo galat nikle to **peeche jaake doosra path** try karo. Jaise maze solve karna: aage badho, dead-end mile to wapas aake doosra raasta try karo.

Stack & Queue Based Problems

Structure	Rule	Real Example	Python
Stack	LIFO (Last In First Out)	Browser back button, undo feature	list.append() / list.pop()
Queue	FIFO (First In First Out)	Printer queue, ticket line	collections.deque

Greedy Thinking

Greedy approach mein har step par wo choice lete hain jo **us waqt sabse best lage** (bina future soche). Jaise coin change problem mein hamesha sabse badi coin choose karna (jab possible ho).

Basic Dynamic Programming (DP)

DP tab use hota hai jab ek problem mein **same sub-problems baar-baar** solve ho rahe hon (jaise Fibonacci). Hum unka answer **store (memoize)** kar lete hain taaki dobara calculate na karna pade.

```
# Fibonacci without DP (slow, repeats work):
def fib(n):
    if n <= 1: return n
    return fib(n-1) + fib(n-2)

# Fibonacci WITH DP (fast, memoization):
memo = {}
def fib_dp(n):
    if n <= 1: return n
    if n in memo: return memo[n]          # already calculated? reuse it!
    memo[n] = fib_dp(n-1) + fib_dp(n-2)
    return memo[n]
```

Binary Search

Sorted list mein element dhundhne ka fastest tareeka — $O(\log n)$. Har step mein **aadhi list discard** kar dete hain.

```
def binary_search(arr, target):
    low, high = 0, len(arr)-1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1          # right half mein dhundo
        else:
            high = mid - 1        # left half mein dhundo
    return -1

print(binary_search([2,4,6,8,10,12], 10))  # Output: 4
```

Prefix Sum

Agar baar-baar kisi range ka sum poochha jaaye, to pehle se ek **prefix sum array** bana lo — phir har query $O(1)$ mein answer ho jaati hai.

```
nums = [2, 4, 6, 8, 10]
prefix = [0]*(len(nums)+1)
for i in range(len(nums)):
    prefix[i+1] = prefix[i] + nums[i]

# Sum from index 1 to 3 (inclusive):
print(prefix[4] - prefix[1])  # 4+6+8 = 18
```

Practice Questions — Level 6

EASY

1. Binary search khud implement karo (loop se)
2. Factorial recursion se likho

MEDIUM

1. Two pointers se check karo ki string palindrome hai ya nahi
2. Sliding window se longest substring without repeating characters dhundo

HARD

1. Fibonacci ko memoization ke saath likho aur bina memoization wale se speed compare karo
2. N-Queens jaisa simple backtracking problem try karo (4-Queens)

Yaad rakhna: pehle khud solve karne ki koshish karo, seedha solution mat dekho!

LEVEL 7

Data Structures & Algorithmic Thinking

Sahi data structure choose karne ka logic

Bahut saari galtiyan tab hoti hain jab hum **galat data structure** choose kar lete hain. Yahan dekhte hain har structure ka use-case, wrong approach vs better approach ke saath.

Structure	Kab Use Karein	Real Example
List	Order matter karta hai, duplicates allowed hain, index se access chahiye	Shopping cart items, history of actions
Set	Sirf unique values chahiye, order matter nahi karta, fast membership check	Unique visitors, duplicate remove karna
Dictionary	Key-value pairs, fast lookup by key chahiye	Phonebook (name→number), frequency counter
Tuple	Fixed, unchangeable data (immutable)	Coordinates (x,y), RGB color values
Stack	Last added item pehle nikalna hai (LIFO)	Undo/Redo, function call stack, bracket matching
Queue	Pehle aaya, pehle jaayega (FIFO)	Task scheduling, printer queue, BFS traversal

Example: Wrong Approach vs Better Approach

Problem: Check karo ki ek list mein koi element pehle se maujood hai ya nahi (bahut baar poochhna hai).

```
# WRONG (Slow) Approach -- List mein 'in' check O(n) hai, har baar poori list scan hoti hai
items = [1, 2, 3, ..., 10000]
if 9999 in items:      # slow for large lists, especially in a loop
    print("Found")

# BETTER Approach -- Set mein 'in' check O(1) hai (average case)
items_set = set(items)
if 9999 in items_set:  # fast, konstant time
    print("Found")

# WHY? Set internally hashing use karta hai, isliye direct location pe check ho jaata hai.
# List mein ek-ek element compare karna padta hai.
```

Practice Questions — Level 7

EASY

1. Batao kis structure ka use karoge: 'saare unique students ke naam store karne ke liye'
2. Batao kis structure ka use karoge: 'ek game ka undo history'

MEDIUM

1. Ek phonebook app banao jisme dictionary use ho

2. Task scheduler banao jisme queue (collections.deque) use ho

HARD

1. Ek Stack aur Queue dono khud class bana ke implement karo (bina built-in ke)

Yaad rakhna: pehle khud solve karne ki koshish karo, seedha solution mat dekho!

LEVEL 8

Universal Problem-Solving Framework

Isse kisi bhi programming problem par apply karo

Step 1: What is the problem? — Problem statement ko apne words mein ek line mein likho

Step 2: What are the inputs? — Konsi values di gayi hain, konsa data type

Step 3: What should be the output? — Result kaisa dikhna chahiye

Step 4: What are the constraints? — Kitna bada input ho sakta hai? Negative allowed hai? Time limit?

Step 5: Can I solve it manually? — Kaagaz par 2-3 examples khud solve karke dekho

Step 6: Can I break it into smaller problems? — Bada problem chhote parts mein todo

Step 7: What data structure should I use? — List/Set/Dict/Stack/Queue — Level 7 wali thinking use karo

Step 8: What algorithm should I use? — Loop/Recursion/Two Pointers/Sliding Window/Binary Search etc.

Step 9: Can I write pseudocode? — Bina syntax ke apna logic likho

Step 10: Can I dry run it? — Table bana ke manually trace karo

Step 11: Can I code it? — Ab Python mein likho

Step 12: Can I optimize it? — Time/space complexity better ho sakti hai kya?

Framework Applied: 'Find the Missing Number' Problem

Problem: List `[1,2,3,5,6]` mein 1 se 6 ke beech ek number missing hai, use dhundo.

Framework Step	Answer
Problem	1 se N tak ke sequence mein ek missing number dhundna hai
Inputs	List of numbers, N (expected range)
Output	Ek number (jo missing hai)
Constraints	Sirf ek number missing hai, baaki sab unique hain
Manual solve	$1+2+3+4+5+6 = 21$, list ka sum = $1+2+3+5+6 = 17$, difference = 4 (missing!)
Data Structure	Sirf list, extra structure ki zaroorat nahi
Algorithm	Expected sum (formula: $N*(N+1)/2$) minus actual sum = missing number

```
nums = [1, 2, 3, 5, 6]
N = 6
expected_sum = N * (N + 1) // 2
actual_sum = sum(nums)
missing = expected_sum - actual_sum
print(missing) # Output: 4
```

Notice karo — hume loop likhne ki zaroorat hi nahi padi! Manually solve karke humne ek **formula-based optimized solution** dhoond liya, jo brute-force (har number check karna) se kaafi fast hai.

DRY RUN

Dry Run Training

Code ko manually trace karna seekho

Dry run karne ka matlab hai — **bina computer chalaye**, apne dimaag/kaagaz par code ko line-by-line follow karna aur variables ki value track karna. Ye galtiyan pakadne ka sabse powerful tarika hai.

Example 1 — Simple

```
total = 0
for i in range(1, 6):
    total += i
```

i	total (before)	calculation	total (after)
1	0	0+1	1
2	1	1+2	3
3	3	3+3	6
4	6	6+4	10
5	10	10+5	15

Example 2 — Thoda Mushkil (with condition)

```
result = []
for i in range(1, 8):
    if i % 2 == 0:
        result.append(i * 2)
    else:
        result.append(i)
```

i	i % 2 == 0?	Action	result (after)
1	No	append(1)	[1]
2	Yes	append(4)	[1, 4]
3	No	append(3)	[1, 4, 3]
4	Yes	append(8)	[1, 4, 3, 8]
5	No	append(5)	[1, 4, 3, 8, 5]
6	Yes	append(12)	[1, 4, 3, 8, 5, 12]
7	No	append(7)	[1, 4, 3, 8, 5, 12, 7]

Example 3 — Nested Loop (Difficult)

```

count = 0
for i in range(1, 4):
    for j in range(1, 4):
        if i == j:
            count += 1

```

i	j	i==j?	count (after)
1	1	Yes	1
1	2	No	1
1	3	No	1
2	1	No	1
2	2	Yes	2
2	3	No	2
3	1	No	2
3	2	No	2
3	3	Yes	3

Final Output: count = 3

NO-CODE ZONE

Logic Without Code

Sirf sochna hai — code likhne ki permission nahi!

Neeche diye gaye problems ke liye **sirf** ye steps karo: Problem Understanding → Steps → Algorithm → Pseudocode → Dry Run. **Python code bilkul mat likhna** — jab tak in sab steps ko poora na kar lo!

1. Ek shop mein agar bill Rs.1000 se zyada ho to 10% discount milta hai. Bill amount lekar final amount nikalne ka logic socho (code nahi!).
2. Ek list mein numbers diye hain. Bina Python ke socho -- unme se sirf wo numbers kaise nikaloge jo 3 aur 5 dono se divisible hain?
3. Teen students ke marks diye hain. Unka average nikal kar Pass/Fail decide karne ka poora logic (bina code) likho -- Pass hone ke liye average ≥ 40 chahiye.

Tip: In teeno problems ke liye apni copy par Problem → Understanding → Algorithm → Pseudocode → Dry Run table banao. Jab tumhara pseudocode 100% clear ho jaaye, tabhi Python mein convert karna.

DEBUGGING

Debugging & Logic Errors

Errors ko pehchanna aur fix karna seekho

Error Type	Kab Hota Hai	Example
Syntax Error	Python ke grammar rules todne par	if x = 5: (== ki jagah = use kiya)
Runtime Error	Code chalte waqt kisi invalid operation par	10 / 0 (ZeroDivisionError)
Logical Error	Code chal jaata hai, par galat result deta hai	Average nikalte waqt count se divide na karna
Infinite Loop	Loop ki ending condition kabhi False na ho	while i < 10: print(i) (i kabhi increment nahi hua)
Wrong Condition	Galat comparison operator	>= ki jagah > use karna, boundary case miss ho jaata hai
Off-by-One Error	Loop ek extra ya ek kam baar chalta hai	range(1, N) likhna jab range(1, N+1) chahiye tha

Bug Hunt — In Codes Mein Kya Galat Hai?

Bug 1

```
total = 0
for i in range(1, 10):
    total = i
print(total)
```

Hint: Expected: 1 se 9 tak sum. Socho — kya ye sahi sum de raha hai?

Bug 2

```
i = 1
while i < 5:
    print(i)
```

Hint: Ye code chalao to kya hoga? Socho terminal mein kya dikhega.

Bug 3

```
def check_even(num):
    if num % 2 = 0:
        return True
    return False
```

Hint: Ye function run hoga bhi ya nahi? Line 2 dhyan se dekho.

Answers (khud try karne ke baad hi dekho!):

- Bug 1 (Logical Error):** total = i har baar total ko overwrite kar raha hai, sum nahi ho raha. Fix: total += i

2. **Bug 2 (Infinite Loop):** `i` kabhi increment nahi hota, isliye `i < 5` hamesha True rahega. Fix: loop ke andar `i += 1` add karo
3. **Bug 3 (Syntax Error):** `num % 2 = 0` mein single `'='` hai (assignment), comparison ke liye `'=='` chahiye

PROJECTS

Final Projects

Beginner → Intermediate → Advanced progression

Beginner Projects

- **Calculator** — +, -, *, / operations, function-based design
- **Number Guessing Game** — random number, hints (high/low), attempt counter
- **Grade Calculator** — marks input → grade output using if-elif ladder
- **Simple ATM** — PIN check, balance, deposit/withdraw (Level 5 mein bana chuke ho)

Intermediate Projects

- **Expense Tracker** — add/view/delete expenses, category summary (Level 4 mein bana chuke ho)
- **Contact Book** — dictionary-based, add/search/update/delete contacts
- **Quiz App** — list of questions, score tracking, timer (optional)
- **Student Management System** — add students, update marks, calculate average/grade

Advanced Projects

- **CLI Banking System** — multiple accounts, login system, transaction history, interest calculation
- **Inventory Management System** — stock tracking, low-stock alerts, sales report
- **Expense Tracker with File Storage** — JSON/CSV file mein data save karna, persistent across runs
- **Mini API-oriented Backend Project** — functions ko 'endpoints' ki tarah design karna (request → process → response)

Har project ke liye follow karo: Requirements → Problem Breakdown → Data Required → Functions Required → Algorithm → Pseudocode → Flow of Logic → Code → Testing → Edge Cases → Improvements (bilkul jaisa Level 5 ke ATM project mein kiya).

ROADMAP

15-Day Logic Building Plan

Ek structured daily practice roadmap

Day	Focus	Kya Karna Hai
Day 1	Foundation	Logic kya hai, problem-solving process, variables, basic thinking
Day 2	Conditions	if/elif/else, nested if, AND/OR/NOT — Level 1 ke saare problems solve karo
Day 3	Loops Basics	for/while, counter, accumulator — 1 to N, sum, factorial
Day 4	Number Problems	Reverse number, palindrome, prime, Armstrong, perfect number
Day 5	Pattern Problems	Star patterns, number patterns — 5+ patterns khud banao
Day 6	Strings	Reverse, palindrome, frequency counter, anagram check
Day 7	Lists	Traversal, max/min, second largest, remove duplicates
Day 8	Dictionaries & Sets	Frequency counting, unique elements, lookups
Day 9	Functions	Modular thinking, parameters/return — Expense Tracker banao
Day 10	Mini Project 1	ATM Simulation ya Number Guessing Game poora banao
Day 11	Debugging Practice	5 buggy codes khud fix karo, error types pehchano
Day 12	Searching & Sorting	Linear/Binary search, Bubble sort khud implement karo
Day 13	Recursion	Factorial, Fibonacci recursion se, base case identify karna practice karo
Day 14	Data Structures Thinking	List vs Set vs Dict vs Stack vs Queue — decision practice
Day 15	Final Project + Framework	Ek naya project khud choose karo, 12-step framework poora apply karo

Tip: Har din kam se kam 3-4 problems khud attempt karo pehle, phir hi is guide ka solution dekho. Agar koi din mushkil lage, to usi topic ko 1 extra din do — speed se zyada **samajhna** important hai.

CHECKLIST

Final Problem-Solving Checklist

Print karke apne desk par lagao!

- ☐ Problem ko 2 baar padha?
- ☐ Apne words mein samjha (explain kiya khud ko)?
- ☐ Input identify kiya (values aur data type)?
- ☐ Expected output identify kiya?
- ☐ Saari conditions/edge cases socha?
- ☐ Problem ko chhote parts mein toda?
- ☐ Required variables list kiye?
- ☐ Algorithm (steps ka sequence) likha?
- ☐ Pseudocode likha (bina Python syntax ke)?
- ☐ Dry run kiya (table bana ke 2-3 examples check kiye)?
- ☐ Sahi data structure choose kiya (List/Set/Dict/Stack/Queue)?
- ☐ Python code likha?
- ☐ Normal inputs se test kiya?
- ☐ Edge cases test kiye (empty, zero, negative, duplicate)?
- ☐ Optimize karne ka socha (time/space better ho sakta hai kya)?

Yaad Rakhna

Coding seekhna ek **skill** hai, talent nahi. Jitna zyada khud sochoge, utna better logic banega. Agar kabhi stuck ho jao, to solution seedha mat dekho — hint lo, ek chhota example khud bana ke try karo, aur phir dobara try karo.

Har problem jo tum khud solve karte ho, wo tumhe agle problem ke liye zyada confident banata hai.

All the best! Keep practicing, keep thinking.